

LEMBAR KERJA PESERTA DIDIK (LKPD) 4

KOLABORASI TIM MENGGUNAKAN GIT & GITHUB

Simulasi Workflow Pengembangan Perangkat Lunak

A. IDENTITAS PEMBELAJARAN

Satuan Pendidikan: SMKN 1 Garut

Program Keahlian: Pengembangan Perangkat Lunak dan Gim

Konsentrasi Keahlian: Rekayasa Perangkat Lunak (RPL)

Mata Pelajaran: Dasar-Dasar Pengembangan Perangkat Lunak dan Gim

Fase/Kelas: F / XI RPL

Materi: Kolaborasi Tim Menggunakan Git dan GitHub

Alokasi Waktu: 2 × 45 menit

Model: Pembelajaran Mandiri Berbasis Proyek

B. IDENTITAS PESERTA DIDIK

Nama: _____

Kelas: _____

Kelompok: _____

Nomor Kelompok: _____

Nama Repository: _____

URL Repository: _____

Nama Project Manager: _____

C. TUJUAN PEMBELAJARAN

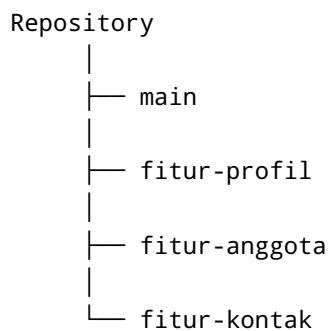
Setelah menyelesaikan LKPD ini, peserta didik mampu:

1. Menjelaskan konsep kolaborasi dalam pengembangan perangkat lunak.
2. Menjelaskan fungsi branch dalam Git.
3. Meng-clone repository GitHub.
4. Membuat dan menggunakan branch.
5. Mengembangkan fitur secara terpisah dari branch utama.

6. Membuat commit yang bermakna.
 7. Melakukan push branch ke GitHub.
 8. Membuat Pull Request.
 9. Melakukan Code Review sederhana.
 10. Memperbaiki kode berdasarkan hasil review.
 11. Melakukan merge Pull Request.
 12. Melakukan sinkronisasi repository menggunakan `git pull`.
 13. Mengidentifikasi konflik sederhana.
 14. Menyelesaikan konflik secara mandiri.
 15. Menerapkan etika kerja kolaboratif dalam pengembangan perangkat lunak.
-

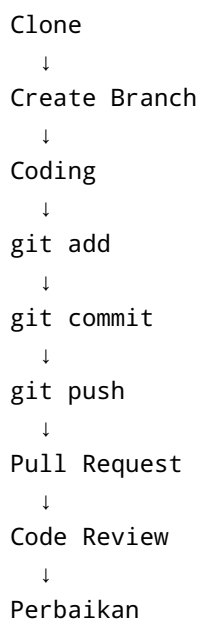
D. KONSEP YANG HARUS DIPAHAMI

Sebelum memulai praktik, pahami hubungan berikut:



Setiap developer bekerja pada branch masing-masing.

Alur kerja:



```
↓  
Merge  
↓  
git pull
```

Jangan mengerjakan fitur langsung pada `main`.

E. SKENARIO INDUSTRI

Bayangkan kelompok kalian adalah sebuah tim developer pada perusahaan software.

Kalian mendapatkan proyek:

"Website Profil Kelas XI RPL"

Klien meminta website sederhana yang menampilkan:

- Profil kelas
- Daftar anggota
- Informasi proyek
- Kontak
- Dokumentasi proyek

Project Manager memberikan aturan:

"Setiap developer wajib mengerjakan fitur melalui branch masing-masing. Tidak diperbolehkan melakukan perubahan langsung pada `main`."

Setelah fitur selesai:

Developer → Pull Request → Code Review → Perbaikan → Merge

Kalian akan mensimulasikan workflow tersebut.

F. PEMBAGIAN PERAN

Bentuk kelompok terdiri dari 3–4 orang.

Jika terdiri dari 4 orang:

Anggota 1 – Project Manager

Tanggung jawab:

- Membuat repository.

- Menentukan pembagian tugas.
- Memeriksa Pull Request.
- Melakukan Code Review.
- Melakukan Merge.

Anggota 2 – Developer Profil

Tugas:

profil.html

Anggota 3 – Developer Anggota

Tugas:

anggota.html

Anggota 4 – Developer Kontak

Tugas:

kontak.html

Semua anggota tetap bertanggung jawab terhadap hasil proyek.

G. ATURAN KERJA

1. Setiap anggota wajib memiliki branch.
 2. Jangan mengubah `main` secara langsung.
 3. Setiap fitur minimal memiliki satu commit.
 4. Pesan commit harus jelas.
 5. Setiap fitur dikirim melalui Pull Request.
 6. Pull Request harus melalui review.
 7. Pull Request yang memiliki kesalahan harus diperbaiki.
 8. Setiap anggota wajib memberikan kontribusi.
 9. Semua anggota wajib memahami workflow yang digunakan.
 10. Jangan menghapus repository ketika mengalami error.
-

H. CHALLENGE 1

Membuat Repository

Project Manager membuat repository baru di GitHub.

Gunakan nama:

website-profil-xi-rpl-[nomor-kelompok]

Contoh:

website-profil-xi-rpl-03

Repository dibuat sebagai:

Public

Tambahkan:

README.md

Isi awal:

```
# Website Profil XI RPL

Website ini merupakan proyek pembelajaran
kolaborasi Git dan GitHub.

## Anggota Tim

1. Nama - Project Manager
2. Nama - Developer
3. Nama - Developer
4. Nama - Developer
```

CHECKPOINT 1

- [] Repository berhasil dibuat
- [] Repository bersifat Public
- [] README tersedia
- [] Semua anggota sudah dicantumkan

URL repository:

I. CHALLENGE 2

Clone Repository

Semua anggota melakukan clone.

```
git clone URL_REPOSITORY
```

Masuk ke folder:

```
cd nama-repository
```

Periksa:

```
git status
```

Pertanyaan

Apa arti hasil `git status` ?

CHECKPOINT 2

- ☐ Repository berhasil di-clone
- ☐ Folder dapat dibuka
- ☐ `git status` berhasil dijalankan

J. CHALLENGE 3

Membuat Branch

Setiap anggota membuat branch.

Developer Profil

```
git switch -c fitur-profil
```

Developer Anggota

```
git switch -c fitur-anggota
```

Developer Kontak

```
git switch -c fitur-kontak
```

Periksa:

git branch

Branch aktif ditandai dengan:

*

Catat:

Nama	Branch

Pertanyaan Analisis

Mengapa setiap developer tidak langsung bekerja pada `main`?

K. CHALLENGE 4

Mengembangkan Fitur

1. Developer Profil

Buat:

profil.html

Minimal berisi:

- Nama kelas
- Nama sekolah
- Program keahlian
- Deskripsi kelas
- Visi kelas

Contoh struktur:

```
<!DOCTYPE html>
<html>
<head>
  <title>Profil XI RPL</title>
</head>
<body>

<h1>XI Rekayasa Perangkat Lunak</h1>

<h2>SMKN 1 Garut</h2>

<p>
Kami adalah siswa kelas XI RPL
yang mempelajari pengembangan perangkat lunak.
</p>

</body>
</html>
```

L. DEVELOPER ANGGOTA

Buat:

```
anggota.html
```

Tampilkan:

- Nama anggota
- Jabatan/peran
- Deskripsi singkat

Contoh:

```
<h1>Anggota Tim</h1>

<ul>
  <li>Nama 1 - Project Manager</li>
  <li>Nama 2 - Developer</li>
  <li>Nama 3 - Developer</li>
</ul>
```

M. DEVELOPER KONTAK

Buat:

```
kontak.html
```

Minimal berisi:

- Email
- Instagram
- Alamat sekolah

Contoh:

```
<h1>Kontak</h1>

<p>Email: xi.rpl@example.com</p>

<p>Instagram: @xi_rpl</p>

<p>SMKN 1 Garut</p>
```

N. CHALLENGE 5

Commit

Setelah fitur selesai:

```
git status
```

Kemudian:

```
git add .
```

Lakukan commit.

Contoh:

```
git commit -m "Menambahkan halaman profil"
```

Contoh lainnya:

```
git commit -m "Menambahkan daftar anggota"
```

```
git commit -m "Menambahkan halaman kontak"
```

Pertanyaan

Apa perbedaan pesan commit berikut?

```
git commit -m "update"
```

dan:

```
git commit -m "Menambahkan halaman profil kelas"
```

Mana yang lebih baik?

O. CHALLENGE 6

Push Branch

Push branch ke GitHub:

```
git push -u origin NAMA_BRANCH
```

Contoh:

```
git push -u origin fitur-profil
```

Buka GitHub.

Pastikan branch muncul.

CHECKPOINT 3

- [] Branch muncul di GitHub
 - [] File fitur muncul
 - [] Commit terlihat
 - [] Tidak ada perubahan langsung pada `main`
-

P. CHALLENGE 7

Pull Request

Developer membuat Pull Request.

Buka:

Pull Requests → New Pull Request

Pilih:

```
base: main
compare: fitur-profil
```

Judul:

Menambahkan halaman profil kelas

Deskripsi:

```
## Perubahan
- Menambahkan profil kelas
- Menambahkan informasi program keahlian

## Tujuan
Menambahkan halaman profil untuk website XI RPL.
```

Klik:

Create Pull Request

Q. CHALLENGE 8

CODE REVIEW

Project Manager bertugas melakukan review.

Periksa:

Struktur

- ☐ Nama file benar
- ☐ HTML memiliki struktur yang benar
- ☐ Tidak ada kode yang tidak diperlukan

Isi

- ☐ Informasi lengkap
- ☐ Tidak ada kesalahan penulisan
- ☐ Sesuai dengan tugas

Kualitas

- ☐ Kode mudah dibaca
- ☐ Indentasi cukup rapi
- ☐ Tidak ada informasi yang tidak relevan

TULISKAN HASIL REVIEW

Nama Developer:

Hasil Review:

- ☐ Approve
- ☐ Request Changes

Komentar Reviewer:

R. CHALLENGE 9

MELAKUKAN PERBAIKAN

Jika reviewer memilih:

Request Changes

Developer membaca komentar.

Lakukan perbaikan pada branch yang sama.

Kemudian:

```
git add .
```

```
git commit -m "Memperbaiki hasil code review"
```

```
git push
```

Pull Request akan diperbarui otomatis.

Project Manager melakukan review kembali.

S. CHALLENGE 10

MERGE

Jika Pull Request sudah memenuhi kriteria:

Project Manager memilih:

Approve → Merge Pull Request

Kemudian pastikan:

- [] Pull Request berhasil di-merge
 - [] Fitur masuk ke `main`
 - [] Tidak ada perubahan yang hilang
-

T. CHALLENGE 11

SINKRONISASI

Setelah fitur anggota lain berhasil di-merge:

Setiap anggota menjalankan:

```
git switch main
```

Kemudian:

```
git pull origin main
```

Periksa apakah file berikut sudah tersedia:

```
profil.html  
anggota.html  
kontak.html
```

U. PERTANYAAN ANALISIS

1. Apa fungsi `git pull`?

2. Apa yang terjadi jika programmer tidak melakukan `git pull`?

3. Mengapa `main` harus dijaga agar tetap stabil?

V. CHALLENGE 12

SIMULASI CONFLICT

Sekarang lakukan tantangan tambahan.

Tujuan tantangan ini adalah memahami bahwa konflik merupakan hal normal dalam pengembangan perangkat lunak.

Dua anggota mengubah bagian yang sama pada `README.md`.

Developer A

Menambahkan:

Website dikembangkan menggunakan HTML.

Developer B

Menambahkan pada bagian yang sama:

Website dikembangkan menggunakan HTML5.

Lakukan commit pada masing-masing branch.

Kemudian buat Pull Request.

Amati apakah GitHub mendeteksi konflik.

W. JIKA TERJADI CONFLICT

Git dapat menampilkan:

```
<<<<<<< HEAD
versi A
=====
versi B
>>>>>>> branch
```

Tugas developer:

1. Membaca kedua perubahan.
2. Menentukan isi yang benar.
3. Menghapus tanda conflict.
4. Menyimpan file.
5. Melakukan commit.
6. Push kembali.

X. PERTANYAAN CONFLICT

1. Mengapa conflict terjadi?

2. Apakah conflict berarti Git rusak?

3. Siapa yang harus menentukan versi kode yang benar?

4. Mengapa komunikasi antar programmer penting?

Y. TROUBLESHOOTING MANDIRI

Jika mengalami error, **jangan langsung menghapus repository.**

Ikuti prosedur:

Langkah 1

Baca pesan error.

Langkah 2

Salin bagian penting dari error.

Langkah 3

Cari penyebabnya.

Contoh:

```
git push rejected non-fast-forward
```

Cari:

```
How to fix git push rejected non-fast-forward
```

Langkah 4

Diskusikan dengan anggota kelompok.

Langkah 5

Gunakan AI jika diperlukan.

Berikan informasi:

Saya sedang belajar Git.

Saya menjalankan:
`git push origin fitur-profil`

Kemudian muncul error:
[pesan error]

Kondisi repository:
[jelaskan kondisi]

Z. TROUBLESHOOTING LOG

No	Error/Masalah	Penyebab	Solusi	Berhasil
1				
2				
3				

AA. FINAL CHECKLIST

Sebelum mengumpulkan, pastikan:

Repository

- ☐ Repository Public
- ☐ Nama repository benar
- ☐ README tersedia
- ☐ Semua anggota tercantum

Git

- ☐ Setiap anggota memiliki branch
- ☐ Tidak ada pekerjaan langsung di main
- ☐ Commit memiliki pesan yang jelas
- ☐ Branch berhasil di-push

GitHub

- ☐ Pull Request dibuat
- ☐ Pull Request direview
- ☐ Ada komentar review
- ☐ Ada perbaikan jika diperlukan

- [] Pull Request di-merge

Produk

- [] profil.html
- [] anggota.html
- [] kontak.html
- [] README.md

Kolaborasi

- [] Semua anggota berkontribusi
 - [] Pembagian tugas jelas
 - [] Masalah didiskusikan
 - [] Troubleshooting dicatat
-

AB. BUKTI Pengerjaan

Setiap kelompok mengumpulkan screenshot:

Bukti 1

Repository GitHub

Bukti 2

Daftar branch

Bukti 3

Commit

Bukti 4

Pull Request

Bukti 5

Code Review

Bukti 6

Merge

Bukti 7

Hasil akhir `main`

URL repository:

AC. REFLEKSI INDIVIDU

Jawablah berdasarkan pengalaman kalian sendiri.

1. Apa perbedaan bekerja sendiri dengan bekerja menggunakan Git dan GitHub?

2. Apa manfaat branch?

3. Mengapa Pull Request diperlukan?

4. Apa manfaat Code Review?

5. Error apa yang paling sulit kalian selesaikan?

6. Bagaimana kalian menemukan solusinya?

7. Apa kontribusi terbesar kalian dalam kelompok?

8. Jika menjadi programmer profesional, kebiasaan apa dari kegiatan ini yang akan kalian pertahankan?

AD. PENILAIAN

1. Keterampilan Git & GitHub — 40%

Kompetensi	Bobot
Clone repository	5
Membuat branch	5
Commit	5
Push	5
Pull Request	10
Code Review	5
Merge & Pull	5

2. Produk — 20%

Aspek	Bobot
Kelengkapan fitur	10
Kerapian kode	5
Dokumentasi	5

3. Kolaborasi — 25%

Aspek	Bobot
Pembagian tugas	5
Kontribusi	10
Komunikasi	5
Tanggung jawab	5

4. Refleksi & Problem Solving — 15%

Aspek	Bobot
Pemahaman konsep	5
Kemampuan troubleshooting	5
Refleksi	5

TOTAL: 100

AE. REFLEKSI AKHIR

Lengkapi kalimat berikut:

Sebelum belajar GitHub, saya berpikir bahwa...

Setelah melakukan kolaborasi dengan GitHub, saya memahami bahwa...

Kesalahan/error yang saya alami mengajarkan saya bahwa...

Jika saya bekerja sebagai programmer dalam sebuah tim, saya akan...

AF. PESAN UNTUK PESERTA DIDIK

"JANGAN TAKUT ERROR."

Dalam dunia pengembangan perangkat lunak, error bukan tanda bahwa kalian gagal.

Error adalah informasi.

Programmer profesional tidak selalu mengetahui jawabannya.

Yang membedakan programmer adalah kemampuannya untuk:

Membaca masalah → mencari informasi → mencoba solusi → menguji → memperbaiki → mendokumentasikan.

Hari ini kalian tidak sedang sekadar belajar GitHub.

Kalian sedang berlatih **cara bekerja sebagai developer dalam sebuah tim.**

Selamat bekerja, berdiskusi, bereksperimen, dan memecahkan masalah.